CEUE203
25IT037

PRACTICAL : 07

Program - 1: Form Validator

Aim :

To understand custom annotations and Java reflection by developing three practice programs: a form validator, a mini test runner, and a column-to-object mapper.

Code :

```
import java.lang.annotation.*;

import java.lang.reflect.Field;

import java.util.ArrayList;

import java.util.List;


@Retention(RetentionPolicy.RUNTIME)

@Target(ElementType.FIELD)

@interface NotBlank { }


@Retention(RetentionPolicy.RUNTIME)

@Target(ElementType.FIELD)

@interface MaxLength {

    int value();

}


class SignupForm {

    @NotBlank
```

CEUE203
25IT037

```
    private String name;

    @NotBlank
    @MaxLength(30)
    private String email;

    @NotBlank
    @MaxLength(12)
    private String password;

    SignupForm(String name, String email, String password) {
        this.name = name;
        this.email = email;
        this.password = password;
    }
}

class FormValidator {
    public static List<String> validate(Object obj) {
        List<String> errors = new ArrayList<>();

        for (Field field : obj.getClass().getDeclaredFields()) {
            field.setAccessible(true);

            try {
                Object value = field.get(obj);

                if (field.isAnnotationPresent(NotBlank.class)) {
```

```
                if (value == null || value.toString().trim().isEmpty()) {
                    errors.add(field.getName() + " must not be blank");
                }
            }

            if (field.isAnnotationPresent(MaxLength.class)) {
                MaxLength rule = field.getAnnotation(MaxLength.class);

                if (value != null &&
                    value.toString().length() > rule.value()) {
                    errors.add(field.getName() +
                        " length must not exceed " + rule.value());
                }
            }
        } catch (IllegalAccessException e) {
            errors.add("Cannot access field: " + field.getName());
        }
    }
    return errors;
  }
}

public class FormValidatorDemo {
  public static void main(String[] args) {
    SignupForm form = new SignupForm(
      "", "student@example.com",
      "thispasswordisverylong"
    );
```

```
        List<String> errors = FormValidator.validate(form);


        if (errors.isEmpty()) {
            System.out.println("Form is valid.");
        } else {
            System.out.println("Validation Errors:");


            for (String error : errors) {
                System.out.println(error);
            }
        }
    }
}
```

Output:

Validation Errors:

name must not be blank

password length must not exceed 12

Program - 2: Mini Test Runner

Aim :

Create a @Run annotation. Use reflection to find and invoke only annotated no-argument methods and count how many methods ran.

CEUE203
25IT037

Code:

```
import java.lang.annotation.*;
import java.lang.reflect.Method;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
@interface Run { }

class SampleTests {

    @Run
    public void testAddition() {
        System.out.println("testAddition executed");
    }

    @Run
    public void testSubtraction() {
        System.out.println("testSubtraction executed");
    }

    public void helperMethod() {
        System.out.println("This method should not run");
    }
}

public class MiniTestRunner {
    public static void main(String[] args) throws Exception {
```

```
        SampleTests tests = new SampleTests();

        int count = 0;


        for (Method method :
                SampleTests.class.getDeclaredMethods()) {


            if (method.isAnnotationPresent(Run.class)
                    && method.getParameterCount() == 0) {


                method.invoke(tests);
                count++;
            }
        }


        System.out.println("Total test methods run: " + count);
    }
}
```

Output:

```
testAddition executed
testSubtraction executed
Total test methods run: 2
```

CEUE203
25IT037

Program 3: Column Mapping Using Reflection

Aim:

To create a custom @Column annotation and use Java Reflection to map header and data row values into object fields.

Code:

```
import java.lang.annotation.*;
import java.lang.reflect.Field;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface Column {
    String name();
}

class Student {

    @Column(name = "student_name")
    private String name;

    @Column(name = "student_age")
    private int age;

    @Column(name = "student_city")
    private String city;
```

CEUE203
25IT037

```
    public String toString() {
        return "Name: " + name +
               ", Age: " + age +
               ", City: " + city;
    }
}

public class ColumnMapperDemo {

    public static void main(String[] args) throws Exception {

        String[] header = {
            "student_name", "student_age"
        };

        String[] row = {"Rahul", "20"};

        Student student = new Student();

        mapRow(student, header, row);

        System.out.println(student);
    }

    static void mapRow(Object obj, String[] header,
                       String[] row) throws Exception {

        for (Field field :
```

```
        obj.getClass().getDeclaredFields()) {


    if (!field.isAnnotationPresent(Column.class)) {
        continue;
    }

    Column column = field.getAnnotation(Column.class);
    int index = -1;

    for (int i = 0; i < header.length; i++) {
        if (header[i].equals(column.name())) {
            index = i;
            break;
        }
    }

    if (index == -1) {
        System.out.println("Missing column: "
                + column.name()
                + " (field left unchanged)");
        continue;
    }

    field.setAccessible(true);
    String value = row[index];

    if (field.getType() == String.class) {
        field.set(obj, value);
```

```
        } else if (field.getType() == int.class) {

            field.setInt(obj, Integer.parseInt(value));

        }

    }

  }

}
```

Output:

Missing column: student_city (field left unchanged)

Name: Tirth, Age: 20, City: null